



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
ICS2121 – MÉTODOS DE OPTIMIZACIÓN

Apuntes de Métodos de Optimización

*Por Francisco García Aubert y Jorge Vera Andreo
Material en desarrollo. No compartir fuera del curso.
(versión 7, Agosto 2025)*



Índice general

8. Heurísticas	3
8.1. Heurísticas Generales: Metaheurísticas	3
8.2. Simulated Annealing	4
8.2.1. Ejemplo: localización de centros de distribución y asignación de clientes	5
8.2.2. Ejemplo: Asignación de trabajos a máquinas	6
8.3. Tabu Search	7
8.4. Algoritmos Genéticos	7
8.4.1. Ejemplo: problema de localización	9
8.5. Comentarios Finales	9

Capítulo 8

Heurísticas

En muchos problemas de optimización requiero resolver problemas difíciles, que se pueden resolver con exactitud en horas o días, cuando en realidad se requiere tener un resultado en cosa de segundos o minutos. Tal es el caso del problema del vendedor viajero (PVV), cuando se debe rutear camiones desde un depot a abastecer clientes en una ciudad. Este tipo de problemas se pueden resolver de manera aproximada, obteniendo buenas soluciones en tiempos cortos. Esto es lo que hacen las heurísticas.

Ejemplo de heurísticas famosas son *Greedy*, 2-Opt o 3-Opt en el PVV. Estos últimos corresponden a tomar dos o tres arcos no consecutivos de un circuito, borrarlos y agregar nuevas combinaciones de arcos que cierren el circuito, a ver si eso disminuye el costo. Esto es un procedimiento rápido que si se aplica sobre muchos pares o tríos de arcos, se pueden obtener buenas soluciones de manera rápida. Si bien no son soluciones óptimas, son soluciones buenas y prácticas para usarla.

Desarrollar heurísticas específicas cada vez que tenemos un problema específico es un buen camino, es lo que se tiene que hacer para las heurísticas lagrangeanas, por ejemplo. Pero también se han desarrollado conceptos generales que permiten crear heurísticas para “casi cualquier problema”. Estos procedimientos son llamados “Metaheurísticas”:

- Procedimientos de estructura general que se pueden aplicar a muchos problemas distintos.
- Aunque su implementación requiere de especificaciones según el problema.

8.1. Heurísticas Generales: Metaheurísticas

Estructura básica de todos estos procedimientos: “búsqueda local”. Consideremos el siguiente problema de optimización general:

$$\min_{x \in C} f(x)$$

donde C es el conjunto de soluciones factibles del problema, el cual asumimos, típicamente, que es discreto. C va a ser nuestro “espacio de búsqueda”. Dado $x \in C$ se define, según el problema, un conjunto $V(x) \subset C$ el cual llamamos una “vecindad” de x . La “vecindad” se obtiene mediante una modificación “marginal” de la solución actual, lo que se puede hacer con una regla computacional que da vecinos de alguna solución, tal como mover dos arcos de un circuito haciendo 2-Opt.

Algoritmo genérico de una Metaheurística

Data: Sea $x^0 \in C, k = 0$.

while *Criterio de parada no se satisface* **do:**

 Elegir $x^{k+1} \in V(x^k)$ tal que $f(x^{k+1}) < f(x^k)$.

$k = k + 1$

end

El elemento clave es cómo construir la vecindad, y esto depende del problema específico. Por ejemplo: el proceso 2-Opt para el PVV es una heurística de búsqueda local.

El problema con la búsqueda local es que podemos quedar “atrapados” en un mínimo local si la etapa 1 no encuentra un punto mejor. Hay varias formas de resolver eso, algunas implican violar el criterio básico de selección:

$$f(x^{k+1}) < f(x^k)$$

Dos procedimientos muy usados para “escaparse” de los mínimos locales son *Simulated Annealing* y *Tabu Search*, que estudiaremos a continuación.

8.2. Simulated Annealing

Simulated Annealing hace una exploración en el espacio de búsqueda, tratando de ir mejorando siempre la solución, pero **de vez en cuando considera aceptar una nueva solución más “mala”** x^{k+1} , es decir si $f(x^{k+1}) \geq f(x^k)$, pero eso se hace con una cierta probabilidad, que es controlada de forma especial a medida que avanzan las iteraciones. En general, cuando comienzo, tengo más probabilidades de cambiarme a soluciones más malas, pero a medida que avanzo, dicha probabilidad va disminuyendo.

Para esto vamos a definir ciertos números $T_0 > T_1 > T_2 > \dots$ tal que $T_i > 0, \forall i$ y $T_i \rightarrow 0$ con $i \rightarrow \infty$. Por ejemplo, una forma típica es $T_{i+1} = \alpha T_i$, con $\alpha < 1$. Si la solución vecina es mejor me cambio de todos modos, pero si no, con cierta probabilidad me muevo.

Algoritmo de Simulated Annealing

Data: Sea $x^0 \in C$, B una constante y T_k una serie de valores que tiende a cero.

while Criterio de parada no se satisface **do:**

 elegir al azar $y \in V(x^k)$

if $f(y) < f(x^k)$:

$x^{k+1} = y, k = k + 1$

else:

 Calcular el valor de la probabilidad de aceptar y :

$$p = e^{-\frac{f(y) - f(x^k)}{BT_k}}$$

if $U(0, 1) \leq p$:

$x^{k+1} = y, k = k + 1$

end

Para aceptar y con probabilidad p , simplemente se genera una uniforme entre 0 y 1 $U(0, 1)$, de modo que si esta es menor que p se acepta y

$$T_k \text{ va disminuyendo} \rightarrow \frac{f(y) - f(x^k)}{BT_k} \text{ va aumentando} \rightarrow e^{-\frac{f(y) - f(x^k)}{BT_k}} \text{ va disminuyendo}$$

La lógica de esto es explorar el espacio de búsqueda, conocer la zona y luego profundizar en buscar buenas soluciones. Para aceptar, simplemente se genera un número aleatorio que distribuye uniforme entre 0 y 1. Si es que p es menor que este número aleatorio generado entonces no se acepta el vecino al que se mueve, en caso contrario si se acepta.

Aquí, B es una constante positiva, la distribución de probabilidad original que debiera usarse es la distribución de Boltzman, usada en Mecánica Estadística. Ahí $B = 1,3806503 \times 10^{-23} [JK^{-1}]$, la constante de Boltzman. El origen de esto está en la inspiración del Simulated Annealing: el proceso físico de “annealing” en metalurgia en el que un metal casi fundido a alta temperatura es enfriado de una manera contralada para obtener una estructura cristalina óptima. El Simulated Annealing fue propuesto por Kirkpatrick en 1983, basado en desarrollos anteriores.

La construcción de vecindades y selección de candidatos se suele hacer en forma aleatoria. La “Temperatura” y otros parámetros se calibran por simulación de MonteCarlo. El criterio de parada en general corresponde al número de iteraciones o cuando ya no hay mejora.

Cápsula de video 8.2.1: Simulated Annealing

Profesor Jorge Vera explica en clases Simulated Annealing, con un ejemplo computacional:
<https://youtu.be/RVhSo-joGdE>

8.2.1. Ejemplo: localización de centros de distribución y asignación de clientes

Vamos a usar como ejemplo el problema de localización-asignación que vimos en el tema de Relajación lagrangeana.

- m posibles puntos donde se pueden instalar centros de distribución
- n clientes que deben ser atendidos desde un único centro de distribución.
- f_i el costo de abrir un centro en i .
- c_{ij} el costo de asignar al cliente j al centro en i .

Pregunta: ¿en qué lugares abrir centros de distribución y a cuáles asignar los clientes? El modelo usa dos tipos de variables:

- y_i , binaria que indica si se abre el centro en i .
- x_{ij} , binaria que indica si el cliente j se asigna al centro i .

El modelo típico es el siguiente:

$$\begin{aligned}
 &\text{mín} \sum_{i=1}^m f_i y_i + \sum_{i=1}^m \sum_{j=1}^n c_{ij} x_{ij} \\
 &\text{s.a.} \quad \sum_{i=1}^m x_{ij} = 1, \quad j = 1, \dots, n \\
 &\quad \sum_{j=1}^n x_{ij} \leq n y_i, \quad i = 1, \dots, m \\
 &\quad y_i, x_{ij} \in \{0, 1\} \quad i = 1, \dots, m; j = 1, \dots, n
 \end{aligned}$$

Una “vecindad” de una solución donde ciertos clientes están conectados a un determinado centro de distribución puede ser cambiar a un cliente se centro, abrir otro centro.

Supongamos estamos actualmente con una solución factible $(\bar{y}, \bar{x})$. Pero podemos hacer algo un poco más estructurado. Notemos que, con y fijos, el problema en los x es un problema “fácil”. Sea:

$$\begin{aligned}
 g(y) &= \text{mín} \sum_{i=1}^m \sum_{j=1}^n c_{ij} x_{ij} \\
 &\text{s.a.} \quad \sum_{i=1}^m x_{ij} = 1, \quad j = 1, \dots, n \\
 &\quad \sum_{j=1}^n x_{ij} \leq n y_i, \quad i = 1, \dots, m \\
 &\quad x_{ij} \in \{0, 1\} \quad i = 1, \dots, m; j = 1, \dots, n
 \end{aligned}$$

Luego, podemos hacer la búsqueda sólo en los y con función objetivo

$$f(y) = \sum_{i=1}^m f_i y_i + g(y)$$

y la evaluación de esto es barata...

Uno de los problemas de “*Simulated Annealing*” es que puede visitar soluciones que ya ha visitado. El ideal sería no volver a visitarlas para ser más eficiente.

8.2.2. Ejemplo: Asignación de trabajos a máquinas

En este problema abordamos la siguiente situación, que se presenta frecuentemente en programación de operaciones en los más diversos ámbitos industriales. Tenemos que realizar n trabajos, los cuales se pueden hacer en cualesquiera de m máquinas. El trabajo j requiere de t_j horas para ser realizado (independientemente de la máquina en que se haga). Si se asignan algunos trabajos a una máquina, esta terminará de trabajar en un tiempo igual a la suma de los tiempos de los trabajos que se asignaron. Ese tiempo, que denotaremos γ_i , para la máquina i , se llama “tardanza” (tardiness, es el término que se usa en Inglés). Existen pesos $w_i > 0$ asociados a cada máquina y queremos encontrar una forma de asignar los trabajos a las distintas máquinas de modo tal que se minimice la “tardanza ponderada total” (total weighted tardiness), dada por $\sum_{i=1}^m w_i \gamma_i$. Adicionalmente, existe el requisito de que cada máquina debe trabajar un tiempo total mínimo de R_{min} horas. Notemos que, una vez asignados algunos trabajos a una máquina, da lo mismo el orden en que se hagan, la tardanza de esa máquina es la misma y la asignación es válida, mientras cumpla el mínimo de horas.

La siguiente figura muestra un ejemplo de 5 trabajos asignados a 2 máquinas (llamadas M1 y M2), con un tiempo mínimo de trabajo $R_{min} = 5$ horas, y con una tardanza ponderada total igual a 69.

$M_1, w_1 = 5$	$t_1 = 2,5$	$t_4 = 8,5$			$\gamma_1 = 11$
$M_2, w_2 = 2$	$t_2 = 2,0$	$t_3 = 2,0$	$t_5 = 3,0$		$\gamma_2 = 7$

Es posible construir un modelo de optimización para el problema. Para esto, definamos variables x_{ij} que valen 1 si el trabajo j es asignado a la máquina i . Se propone el siguiente modelo:

$$\begin{aligned}
 \text{mín} \quad & \sum_{i=1}^m w_i \sum_{j=1}^n t_j x_{ij} \\
 \text{s.a.} \quad & \sum_{j=1}^n t_j x_{ij} \geq R_{min} \quad i = 1, \dots, m \quad (1) \\
 & \sum_{i=1}^m x_{ij} = 1 \quad j = 1, \dots, n \quad (2) \\
 & x_{ij} \in \{0, 1\} \quad i = 1, \dots, m; j = 1, \dots, n \quad (3)
 \end{aligned}$$

La función objetivo minimiza la tardanza ponderada ya que γ_i es, precisamente, igual a $\sum_{j=1}^n t_j x_{ij}$ y no es necesario usar explícitamente las variables γ_i .

El problema anterior es de programación entera y seguramente podría resultar difícil de resolver directamente con software como Gurobi o, incluso, haciendo Generación de Columnas (de hecho, es NP-completo). Pero es posible abordarlo mediante heurísticas y ahora mostramos cómo se podría desarrollar una de tipo Simulated Annealing, para abordar este problema.

Es importante notar, primero, que prácticamente cualquier asignación es factible, mientras cumple con la restricción de tiempo mínimo. La búsqueda se hace en las variables x , pero debe cumplirse con las restricciones

$$\sum_{j=1}^n t_j x_{ij} \geq R_{min}$$

Entonces, si estamos en una solución $\bar{x}$, una forma de generar un vecino puede ser:

1. Elegir una máquina al azar, llamémosla i .
2. Buscar un trabajo j tal que si es removido, se mantiene el tiempo mínimo.
3. Reubicar el trabajo removido en otra máquina (y la restricción de tiempo mínimo se seguirá cumpliendo).
4. Si no se puede encontrar un trabajo que cumpla, buscar en otra máquina.

Podría pasar que esto no encuentre un vecino válido, así que otra posibilidad es buscar tratando de hacer un intercambio de trabajos entre dos máquinas y que sigan preservando las restricciones de tiempo mínimo.

La evaluación de la función objetivo es muy simple, ya que sólo hay que calcular $\sum_{i=1}^m w_i \sum_{j=1}^n t_j x_{ij}$, no es necesario resolver ningún problema de optimización.

8.3. Tabu Search

El procedimiento fue desarrollado por Fred Glover en los 70's. Trata de resolver los problemas que presentan los métodos anteriores, cuando hay riesgo de volver a una solución ya visitada. Para eso vamos a “prohibir” visitar soluciones que ya han sido analizadas, para lo que guardamos una “Lista Tabú”. Además, también vamos a permitir ir a soluciones peores.

En algunos momentos vamos a cambiar completamente la estructura de las soluciones que se están analizando, esto es **diversificación**. El algoritmo mantiene una lista, L , de soluciones “tabú”. De este modo, este algoritmo escapa de los óptimos locales por medio de la aplicación de dos reglas básicas:

- No repetir soluciones visitadas.
- Seguir explorando aunque se empeore la solución actual.

Algoritmo de Tabú Search

```

Data: Sea  $x^0 \in C, k = 0, L = \{\emptyset\}$ .
while Criterio de parada no se satisface do:
    Construir vecindad  $V(x^k)$ 
    Elegir  $y$  solución de

                                 $\min \{f(x) : x \in V(x^k)\}$ 

    if  $y \in L$ :
        Elegir otro  $y$ :

                                 $f(y) < f(x^k), y \notin L$ 

    if no se puede encontrar  $y$  adecuado:
        Aceptar  $y : f(y) > f(x^k)$  con alguna probabilidad
        Aplicar diversificación para generar un nuevo  $y$ 
     $x^{k+1} = y, k = k + 1, L = L \cup \{y\}$ 
end

```

La construcción de $V(x^k)$ se hace, muchas veces, en forma aleatoria y la idea es que sea “simple” y rápido de ejecutar, sin importar necesariamente llegar al óptimo pero si a alguna buena solución en un tiempo decente. Notar que el procedimiento hace una optimización local. Uno de los aspectos claves es el largo de la lista Tabú, el que se calibra usando simulación. Cuando se acepta una solución peor, se puede hacer “al estilo Simulated Annealing”, entre otras. La **diversificación** es una etapa importante, se trata de generar un patrón de solución completamente diferente, es decir, “saltar” a otra región del espacio de búsqueda.

La Búsqueda Tabú ha tenido éxito aplicada a algunos problemas combinatoriales de gran tamaño, por ejemplo, problemas de ruteo vehicular.

8.4. Algoritmos Genéticos

Un conjunto completo de métodos heurísticos se ha clasificado bajo el nombre de “Algoritmos Evolutivos”. La idea es imitar procesos evolutivos o biológicos. Las jirafas tienen el cuello largo por qué la comida estaba alta en los árboles y los individuos con cuellos largos se pudieron alimentar mejor y reproducir mejor, haciendo que la generación posterior tuviera en mayor proporción el cuello más largo. Con mucho tiempo, el cuello les creció más y más según el mismo argumento.

Los algoritmos genéticos trabajan sobre una población de soluciones. Replicando un proceso similar al de la evolución de los seres vivos, el algoritmo reproduce, muta y selecciona a las soluciones más fuertes en busca de mejores soluciones.

La primera etapa es codificar las soluciones factibles del problema en los siguientes elementos:

- Un conjunto G de “genes”.
- Cada solución factible del problema combinatorial es representada por un elemento $(g_1, g_2, \dots, g_p)$, donde $g_i \in G$. Este es un “cromosoma”.
- Se define una función $h(g_1, g_2, \dots, g_p)$, llamada “función de adaptación” (*fitness function*).

Las tres componentes básicas de un algoritmo genético son:

1. **Selección:** Se seleccionan dos soluciones dentro de la población para su reproducción. En esta selección se debe aplicar un criterio de aptitud, de forma de que mejores soluciones tengan mayor probabilidad de reproducirse.
2. **Crossover:** Se diseña un método para obtener a partir de dos soluciones una nueva solución, de forma similar en la que los genes de dos individuos se combinan en un nuevo individuo.
 - Sean $v = (v_1, \dots, v_p) \in G^p$ y $w = (w_1, \dots, w_p) \in G^p$ dos cromosomas.
 - Elegir $k \in \{1, \dots, p\}$ aleatoriamente.
 - Generamos dos nuevos individuos, v' y w' :
 - a) $v' = (w_1, \dots, w_k, v_{k+1}, \dots, v_p)$
 - b) $w' = (v_1, \dots, v_k, w_{k+1}, \dots, w_p)$
 - Conservamos los nuevos individuos si están “mejor adaptados”.
3. **Mutación:** Las nuevas soluciones son alteradas con cierta probabilidad, de forma de obtener nuevos atributos y diversificar la búsqueda. Para esto se hace lo siguiente:
 - Sea $v = (g_1, \dots, g_p) \in G^p$ un cromosoma.
 - Elegir $j \in \{1, \dots, p\}$ aleatoriamente.
 - Reemplazar g_j con $g' \in G, g' \neq g_j$.
 - Esto genera un nuevo individuo cuya “adaptabilidad” es evaluada usando h .
 - Si es mejor que el anterior, se conserva. Si no, se busca otra mutación.

Algoritmos Genéticos

Data: 0. Comenzar con una población $U^0 \subset G^p$.

while *Criterio de parada no se satisface* **do:**

Selección: Escoger aleatoriamente un conjunto $B \subset U$ de individuos “mejor adaptados”, usando h .
Aplicar “operadores genéticos” a B para generar un nuevo conjunto B' con mejores individuos que B :

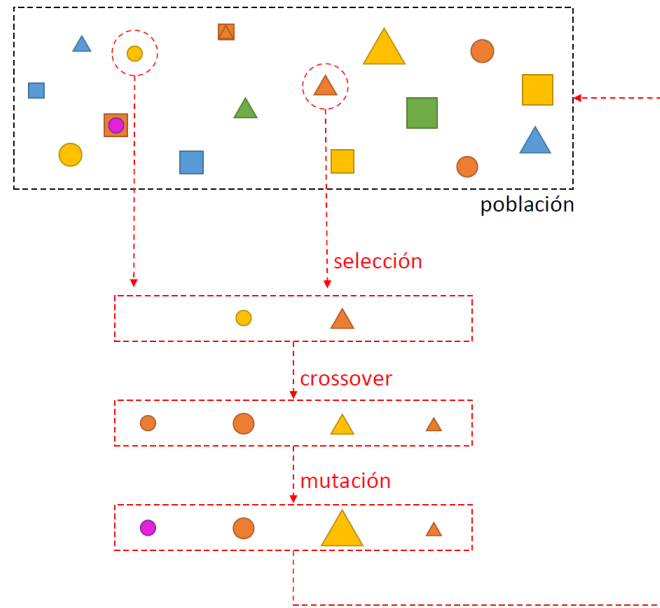
- Reproducción: aplicar *crossover* entre los padres para obtener hijos.
- Mutación: perturbar, con cierta probabilidad baja, las soluciones hijo obtenidas.

Reemplazo: B' reemplaza a B en U .

end

En general, existe una componente aleatoria para seleccionar la población inicial y para elegir los individuos para *Crossovers* y Mutación, dándole mayor probabilidad de elegirse aquellos que son mejor calificados por la función de adaptabilidad. Para la función de adaptabilidad podría usarse la función objetivo original, pero también puede incluir una medida de satisfacción de restricciones, penalizando si estas no se respetan

Los AG se han aplicado exitosamente en algunos problemas relacionados a secuenciamiento (*scheduling*), que son problemas combinatoriales.



8.4.1. Ejemplo: problema de localización

Tomemos nuevamente el problema de Localización y Asignación de clientes. Un Algoritmo Genético se puede desarrollar de muchas formas diferentes. Notemos, que la codificación es "natural" que tenemos variables 0-1. Tomemos el mismo esquema para el problema que usamos en el ejemplo de Búsqueda Tabú: El cromosoma es una solución $y = (y_1, \dots, y_m)$ y los genes son los valores y_i . Como función de adaptación podemos tomar:

$$h(y) = - \left(\sum_{i=1}^m f_i y_i + g(y) \right)$$

La población inicial se genera como un conjunto de "cromosomas", en forma aleatoria, y luego, usar los operadores de mutación y recombinación.

Por ejemplo, en una mutación de $(1, 1, 1, 0, 0, 1, 0)$ puede consistir en lo siguiente:

$$\text{Mutación}\{(1, 1, 1, 0, 0, 1, 0)\} \longrightarrow (1, 0, 1, 0, 0, 1, 0)$$

Y una recombinación de $(1, 1, 1, 0, 0, 1, 0)$ con $(0, 1, 0, 0, 1, 1, 0)$ puede obtenerse mezclando las primeras tres del primero y las últimas cuatro del segundo:

$$\text{Recombinación}\{(1, 1, 1, 0, 0, 1, 0), (0, 1, 0, 0, 1, 1, 0)\} \longrightarrow (1, 1, 1, 0, 1, 1, 0)$$

8.5. Comentarios Finales

- ¿Cuál es la mejor heurística para un problema? Pues, depende. El buen funcionamiento depende mucho de la calibración adecuada de parámetros. Pero hay mucha experiencia práctica que sirve de guía. Por ejemplo, la Búsqueda Tabú ha sido muy buena en problemas "tipo" PVV, ruteo de vehículos y relacionados. Existen gran cantidad de estudio teóricos sobre ciertos procedimientos heurísticos para saber qué tan buenos son. Un punto central: balance entre velocidad de resolución y calidad de la solución.